Services in ASP.NET Core Web API

Services in ASP.NET Core Web API

In ASP.NET Core Web API, controllers are responsible for handling HTTP requests and returning responses. While it is technically possible to write all business logic and data access logic inside controllers, this approach quickly becomes messy and complicated to maintain as the project grows.

To solve this, **Services** are introduced. Services act as dedicated classes that contain the application's **business logic** and **data manipulation code**, keeping controllers clean and focused only on coordinating HTTP requests and responses. This separation of concerns improves readability, maintainability, testability, and scalability of the application.

Problems with Writing Logic Inside Controllers

If we directly embed business logic and data access operations inside controllers, the following problems arise:

- **Code Duplication**: Reusing logic across multiple controllers results in repeated code.
- **Reduced Testability**: Testing controllers becomes more challenging because business rules are intertwined with request-handling logic.
- **Tightly Coupled Code**: Changes in business rules force changes in multiple controllers, reducing flexibility.
- **Violation of the Single Responsibility Principle (SRP)**: Controllers handle both HTTP request/response management and business rules, making them overloaded.

Real-time Analogy: Imagine a restaurant where waiters (controllers) not only take orders but also cook food and manage inventory. It creates chaos. Instead, waiters should only handle customers, while chefs (services) handle cooking.

How Services Solve These Problems

By moving business logic into dedicated service classes, controllers can remain lightweight and focused only on HTTP concerns.

- **Encapsulation of Logic**: Business rules and data access logic live inside services.
- **Reusability**: Services can be shared across multiple controllers.
- **Testability**: Services can be unit-tested independently without worrying about HTTP request handling.
- **Maintainability**: Easier to manage, extend, or replace logic without affecting controllers.

Real-time Analogy: In the restaurant example, services are like chefs in the kitchen. The waiter (controller) only needs to pass the order to the chef and deliver the dish back to the customer. Preparing the food is the chef's (service) responsibility.

Where Should We Write Validation Logic?

Validation in ASP.NET Core Web API can be handled in two ways:

- **At the Controller Level**: ASP.NET Core automatically validates model state if **[ApiController]** is used. Invalid requests return 400 Bad Request automatically.
- **At the Service Level**: Business-specific validations (e.g., "price cannot exceed 10,000" or "product name must be unique") should be placed in services.

Best Practice:

- Use **Data Annotations** in **Models/DTOs** for general input validation (length, required fields, ranges).
- Use **Services** for business logic validations (unique constraints, cross-entity rules).

Real-time Analogy: Think of model validation as security at the gate (basic checks: ID card, ticket) while service validation is like detailed security inside the venue (checking for prohibited items specific to the event).

Creating a Product Service

First, create a folder named **'Services'** at the project root directory, where we will create all service interfaces and their corresponding concrete implementations.

Step 1: Define an Interface (Contracts)

Create an interface named **IProductService.cs** within the **Services** folder, and copy-paste the following code.

```
using MyFirstWebAPIProject.DTOs;
namespace MyFirstWebAPIProject.Services
{
    public interface IProductService
    {
        IEnumerable<ProductDTO> GetAllProducts();
        ProductDTO? GetProductById(int id);
        ProductDTO CreateProduct(ProductCreateDTO createDto);
        bool UpdateProduct(int id, ProductUpdateDTO updateDto);
        bool DeleteProduct(int id);
    }
}
```

Code Explanations:

- Interfaces define a **Contract**. They specify ***what methods exist*** without defining ***how they work***.
- This ensures **Loose Coupling**. The controller doesn't care about the actual implementation, only that the service provides these methods.
- It also enables **testability** (we can mock this interface in unit tests).

Real-time Analogy: Think of an interface as a **Menu Card in a Restaurant**. It lists what you can order (methods) but doesn't tell you how the dish is prepared (implementation).

Step 2: Implement the Service

Create a class file named **ProductService.cs** within the **Services** folder, and copy-paste the following code. The following class implements the **IProductService** interface and provides implementations for all interface methods.

```
using MyFirstWebAPIProject.DTOs;
using MyFirstWebAPIProject.Models;

namespace MyFirstWebAPIProject.Services
{
    public class ProductService : IProductService
    {
        private static List<Category> _categories = new()
        {
            new Category { Id = 1, Name = "Electronics" },
            new Category { Id = 2, Name = "Furniture" },
        };

        private static List<Product> _products = new()
        {
            new Product { Id = 1, Name = "Laptop", Price = 1000.00m, CategoryId =
1 },
            new Product { Id = 2, Name = "Desktop", Price = 2000.00m, CategoryId
= 1 },
            new Product { Id = 3, Name = "Chair", Price = 150.00m, CategoryId = 2
},
        };

        public IEnumerable<ProductDTO> GetAllProducts()
        {
            return _products.Select(p => new ProductDTO
            {
                Id = p.Id,
                Name = p.Name,
                Price = p.Price,
```

```
                CategoryName = _categories.FirstOrDefault(c => c.Id ==
p.CategoryId)?.Name ?? "Unknown"
            });
        }

        public ProductDTO? GetProductById(int id)
        {
            var product = _products.FirstOrDefault(p => p.Id == id);
            if (product == null) return null;

            return new ProductDTO
            {
                Id = product.Id,
                Name = product.Name,
                Price = product.Price,
                CategoryName = _categories.FirstOrDefault(c => c.Id ==
product.CategoryId)?.Name ?? "Unknown"
            };
        }

        public ProductDTO CreateProduct(ProductCreateDTO createDto)
        {
            var newProduct = new Product
            {
                Id = _products.Max(p => p.Id) + 1,
                Name = createDto.Name,
                Price = createDto.Price,
                CategoryId = createDto.CategoryId
            };

            _products.Add(newProduct);

            return new ProductDTO
            {
                Id = newProduct.Id,
                Name = newProduct.Name,
                Price = newProduct.Price,
                CategoryName = _categories.FirstOrDefault(c => c.Id ==
newProduct.CategoryId)?.Name ?? "Unknown"
            };
        }

        public bool UpdateProduct(int id, ProductUpdateDTO updateDto)
        {
            var product = _products.FirstOrDefault(p => p.Id == id);
            if (product == null) return false;
```

```
            product.Name = updateDto.Name;
            product.Price = updateDto.Price;
            product.CategoryId = updateDto.CategoryId;
            return true;
        }

        public bool DeleteProduct(int id)
        {
            var product = _products.FirstOrDefault(p => p.Id == id);
            if (product == null) return false;

            _products.Remove(product);
            return true;
        }
    }
}
```

Code Explanations:

- This class provides the **Actual Implementation** of the interface.
- It encapsulates the **Business Logic** (e.g., creating, updating, and deleting products).
- It isolates **Data Handling** (right now, hardcoded lists; later, database logic).
- If the logic changes (say you move from in-memory list → EF Core database), only the service changes, not the controller.

Real-time Analogy: If the interface is the **menu**, the service is the **kitchen** where chefs actually cook the food.

Step 3: Register the Service in the Program.cs

Next, please add the following code to the Program class file.

```
builder.Services.AddScoped<IProductService, ProductService>();
```

Code Explanations:

- ASP.NET Core uses **Dependency Injection (DI)** by default.
- Here, we register IProductService with its implementation, ProductService.
- AddScoped means a **New Instance** of ProductService will be created for each HTTP request.
- This ensures services are **injected** automatically into controllers wherever required.

Real-time Analogy: This is like telling the **Restaurant Manager**: “Whenever a customer orders something from the menu (IProductService), send it to this specific kitchen (ProductService).”

Step 4: Use the Service in the Controller

Finally, update the Products Controller as follows.

```
using Microsoft.AspNetCore.Mvc;
using MyFirstWebAPIProject.DTOs;
using MyFirstWebAPIProject.Services;

namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductsController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet]
        public ActionResult<IEnumerable<ProductDTO>> GetProducts()
        {
            return Ok(_productService.GetAllProducts());
        }

        [HttpGet("{id}")]
        public ActionResult<ProductDTO> GetProduct(int id)
        {
            var product = _productService.GetProductById(id);
            if (product == null) return NotFound(new { Message = $"Product with
ID {id} not found." });
            return Ok(product);
        }

        [HttpPost]
        public ActionResult<ProductDTO> PostProduct([FromBody] ProductCreateDTO
createDto)
        {
            var createdProduct = _productService.CreateProduct(createDto);
            return CreatedAtAction(nameof(GetProduct), new { id =
createdProduct.Id }, createdProduct);
        }

        [HttpPut("{id}")]
        public IActionResult UpdateProduct(int id, [FromBody] ProductUpdateDTO
updateDto)
        {
```

```
            if (id != updateDto.Id) return BadRequest(new { Message = "ID
mismatch between route and body." });
            if (!_productService.UpdateProduct(id, updateDto)) return
NotFound(new { Message = $"Product with ID {id} not found." });
            return NoContent();
        }

        [HttpDelete("{id}")]
        public IActionResult DeleteProduct(int id)
        {
            if (!_productService.DeleteProduct(id)) return NotFound(new { Message
= $"Product with ID {id} not found." });
            return NoContent();
        }
    }
}
```

Code Explanations:

- The controller **depends** on IProductService, not directly on ProductService.
- ASP.NET Core's DI system automatically provides an instance of ProductService when the controller is created.
- This keeps the controller **clean**:
    - No product management logic inside.
    - Only coordinates between HTTP requests and the service methods.

Real-time Analogy: The **waiter (controller)** doesn't cook the food. He takes the order and passes it to the **chef (service)**, then brings the result back to the customer.

Services in ASP.NET Core Web API provide a clean and maintainable way to separate business logic from controllers. Instead of overloading controllers with both request-handling and core logic, we delegate responsibilities to services. Controllers focus solely on handling HTTP requests and responses, while services encapsulate the actual business rules and data operations. This separation ensures **scalability, maintainability, testability, and cleaner architecture**, making your API robust and production-ready.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

2 thoughts on “Services in ASP.NET Core Web API”

RAYMOND SEMAKULA
OCTOBER 11, 2025 AT 6:56 AM

Hey sir thank you so much for everything but could you make a more detailed tutorial on services okay using more indepth real world problems like calculations. whatever anyway i love this site, it’s the best

Reply

JITENDRA
APRIL 22, 2026 AT 3:22 AM

so many ads even inside code blocks, code is hard to read as it is not aligned because of ad

Reply